CompSci 330: Design and Analysis of Algorithms
Summer 2024, Homework 2
Due: Mon 7/22 @ 6pm, Fri 7/26 @ 11:59pm

Directions

How to Do Homework. We recommend the following three step process for homework to help you learn and prepare for exams.

1. Give yourself 15-20 minutes per problem to try to solve on your own, without help or external materials, as if you were taking an exam. Try to brainstorm and sketch the algorithm for applied problems. Don't try to type anything yet.

2. After a break, review your answers. Lookup resources or get help (from peers, office hours, Ed discussion, etc.) about problems you weren't sure about.

3. Rework the problems, fill in the details, and typeset your final solutions.

Submission. Your written solutions should be typeset and submitted as a single PDF on Gradescope. LaTeX[1] is preferred but not required. If you use another editor for your solutions (such as Microsoft Word), you should convert the final document to a PDF, confirm that the symbolic math from the equation editor is properly formatted, and submit the PDF. You must mark the locations of your solutions to individual problems on Gradescope as explained in the documentation. Implementation problems will request that you submit code separately on Gradescope to be autograded.

Writing Expectations. If you are asked to provide an algorithm, you should clearly and unambiguously define every step of the procedure as a combination of precise sentences in plain English or pseudocode. If you are asked to explain your algorithm, its runtime complexity, or argue for its correctness, your written answers should be clear, concise, and should show your work. Do not skip details but do not write paragraphs where a sentence suffices. Results covered in lecture may be referenced and used directly without any justification of their correctness (unless asked for).

Collaboration. You can (and should!) work with a partner, in which case you will submit a single solution as a group on Gradescope ***by adding your partner to your submission on Gradescope*** (see here for directions). ***Always cite anyone/anything consulted*** outside of course materials, including a brief description of their contribution to your work. See also our policy on collaboration on the course webpage.

Grading. Theory problems will be graded on an S/U scale (for each sub-problem). Applied problems typically have a separate autograder where you can see your score. **The lowest scoring problem is dropped.** See the course assignments webpage for more grading details.

[1] If you are new to LaTeX, you can download it for free at latex-project.org or you can use the popular and free (for a personal account) cloud-editor overleaf.com. We also recommend overleaf.com/learn for tutorials and reference.

Comments for HW3:

- Problems 1 and 2 have ***optional deadlines on Monday @ 6pm***. Feedback will be provided before the review on Tuesday, along with our solutions for those problems. Of course, the solutions should not be shared with other groups before the final deadline on Friday.
- For Problem 1, one should follow our template for DP solutions. These are listed at the start of Discussions 3 and 4 and HW2.
- For Problems 2-4, it is expected that you use reductions to graph problems. ***You are highly encouraged NOT to modify known algorithms for these problems.*** Your solution should involve the following clearly defined:
  1. The vertices of the graph.
  2. The edges of the graph.
  3. Any other information, such as edge weights or labels.
  4. The specific graph problem that you are reducing the given problem to. Write it in terms of the graph, e.g. "compute the shortest path from vertex s to vertex t."
  5. How to solve that graph problem, and how to transform its solution back to a solution for the given problem.
  6. Finally, a runtime analysis in terms of the given input (***NOT*** just the number of vertices and edges of the graph you constructed).
- There is no applied problem for this HW.

Problem 1

Let $G = (V, E)$ be a directed acyclic graph where each vertex $v \in V$ is associated with a number of stars between 1 and 5, denoted by $\bigstar(v) \in \{1, 2, \ldots, 5\}$, and let $s, t \in V$ be distinct vertices.[2] Describe and analyze an $O(V + E)$-time algorithm to compute the shortest path from s to t in G with at least total 330 stars among the vertices in the path, or report that there is no such path. *[Hint: Compute a topological ordering and use it to define a valid evaluation order.]*

Problem 2

Let $G = (V, E)$ be a directed graph where each edge is *colored* grey or Duke blue, denoted by $c(e) \in \{g, b\}$, and let s, t be distinct vertices. Describe a $O(V + E)$-time algorithm to determine if there is a walk in G from s to t where no three consecutive edges have the same color. For example, if the walk has two grey edges in a row, then either the walk must end or the next edge after those two grey edges must be blue. Justify its correctness and analyze its runtime complexity.

[2] In LaTeX, the star symbol ★ is `$\bigstar$`.

Problem 3

Alex is shopping for a new house in another city. Its road network is represented by an undirected graph $G = (V, E)$ with positive edge weights given by $w : E \to \mathbb{R}^+$, where the vertices are points-of-interest, and edges are direct road segments between them, and $w(e)$ for an edge e is the amount of time in minutes it takes to travel down (following the speed limit). (There are no one-way streets, so modeling the edges as undirected suffices.)

Let $H \subset V$ be a subset of houses for sale, let $D \subset V$ be the subset of daycares, and let $p \in V$ be the parking lot for his new job's building. Every weekday, Alex has to drop his son off at a daycare on his way to work, then pick his son up (at the same daycare) on his way home after work. Also interested in minimizing his commute, Alex wants to find the house $h \in H$ with the shortest total driving time each day. That is, the shortest drive from a house $h \in H$ to a daycare $d \in D$, then to p, then back to d, and finally back to h.

Describe an $O(E+V \log V)$-time algorithm to find the desired house for Alex. Justify its correctness and analyze its runtime complexity. *[Hint: First, would Alex use different routes to and from work? Second, ideas from the graph reductions in Discussion 5 may be useful here.*]

Problem 4

There are n airports, each given an index from 1 to n. There are m flights between pairs of airports each day, where at most F flights depart from each airport. Each j-th flight is represented by a 4-tuple (d_j, a_j, t_j, ℓ_j), where:

- $d_j \in \{1, \ldots, n\}$ is the airport from the flight departs,
- $a_j \in \{1, \ldots, n\}$ is the airport at which the flight arrives,
- $t_j \in \{0, 1, \ldots, 1439\}$ is the number of minutes after midnight that the flight departs from airport d_j (e.g., $t_j = 1013$ corresponds to 4:53 PM and $t_j = 1439$ corresponds to 11:59 PM), and
- $\ell_j \in \{0, 1, \ldots, 1439\}$ is the number of minutes after midnight that the flight arrives at airport a on the same day.

Note that, by definition, $0 \le t_j < \ell_j < 1440$, and no flight is midair at midnight; all flights land on the same day that they depart. For example, $(9, 3, 653, 1136)$ represents a flight from airport 9 to airport 3 that departs at 10:53 AM and lands at 6:56 PM.

Suppose you wish to find the earliest time that you can arrive at airport n, assuming you begin at airport 1 at midnight (time 0), by taking a sequence of flights (or possibly a direct one). As in real life, you can only take a flight that departs from an airport i if you are presently at that airport before the flight departs. Describe an algorithm to find this earliest arrival time, whose input is the number of airports, n, and the m tuples representing flights, where at most F flights depart from each airport. Your algorithm should be as fast as you can achieve in terms of n, m, and F.[3] Justify its correctness and analyze its runtime complexity.

To think about later: (a) Suppose you wish to minimize the total time spent *in-flight*, not counting time at airports, to travel from airport 1 to n. (b) Suppose flights come with a 5th value, the price to take the flight, and now you wish to minimize the total *cost* to travel between airports 1 and n.

[3] Because this is a graph reduction problem, the runtime will depend on the size of the graph you construct. First obtain a correct reduction, then consider how to further improve the construction/runtime after.